

Lab Companion — Querying External Data and Iceberg Tables

Companion for the Day 2 (Tue Oct 6) after-lunch lab, 13:00–13:45 (45 min), run from your Skills classroom seat. This is the Bronze-layer lab: you query data in Cloud Storage that is never loaded into BigQuery. The step-by-step instructions live in the lab itself; this file is your orientation, dialect decoder, and gotcha list.

- Classroom: <https://www.skills.google/ilt/classrooms/37990>
- Related notes: [M9 student notes](#) · [Iceberg explained](#) · [M10 governance + medallion notes](#)

What you'll do

From the lab's own introduction:

- Explore how BigQuery's Lakehouse tables can query data in open-source formats directly within Cloud Storage.
- Create a Lakehouse table that points to an existing Apache Iceberg table.
- Analyze data without moving or duplicating it.

The lab has two stretches: first the DDL that points BigQuery at the existing Iceberg data in a Cloud Storage bucket, then ordinary SQL — filters, aggregations — against the table you just created.

Why it matters for your migration

This is your Bronze layer, hands-on. The pattern from your architecture doc: raw data stays in open formats in object storage, and BigQuery queries it in place. Bronze is not a landing zone you copy out of — it is queryable from minute one. In this morning's DIN activity (§30, external tables) you saw files in a bucket become a read-only table with zero loading; this lab takes the same idea and adds Apache Iceberg's table semantics (metadata, ACID behavior) on top of the files. When you design Bronze for the real project, this lab is the reference experience.

Before you start

- **Launch the lab from the Skills classroom** and wait for your seat to provision. You work in your own **lab-provided GCP project** with lab credentials — not any project you may have used before.
- **Check the project picker** at the top of the console before every create. The console sometimes shows a different project from an earlier session; creating the table in the wrong project is the most common self-inflicted error.
- **Have your class dataset ready** — you created it on Day 1 morning (DIN §3). If it's missing, create it now (Explorer pane → three-dot menu next to your project → Create dataset). The table you create needs a dataset to live in.

- If the console asks you to **enable an API** or accept terms along the way, click enable/accept. That is expected in a fresh lab project, not an error.
- Type the DDL yourself at least once rather than pasting it — the muscle memory is the point.

T-SQL → BigQuery SQL: the dialect gotcha box

This is where you'll actually spend your debugging time. BigQuery speaks **GoogleSQL** (standard SQL) — close to T-SQL, different in the places your fingers remember. Keep this box open next to the editor.

Your T-SQL reflex	The GoogleSQL way
USE mydb; then bare table names	No USE. The dataset is part of the name: `project.dataset.table`
[dbo].[Orders] square brackets	Backticks: `class.orders` — square brackets are a syntax error
SELECT TOP 100 ...	SELECT ... LIMIT 100 (at the end of the query)
ISNULL(col, 0)	IFNULL(col, 0) or COALESCE(col, 0)
GETDATE() / GETUTCDATE()	CURRENT_DATE() / CURRENT_TIMESTAMP()
DATEADD(day, -7, col)	DATE_SUB(col, INTERVAL 7 DAY)
DATEDIFF(day, start, end)	DATE_DIFF(end, start, DAY) — argument order flips
YEAR(col) / DATEPART(m, col)	EXTRACT(YEAR FROM col) / DATE_TRUNC(col, MONTH)
String concatenation with +	CONCAT(a, b) or the `
LEN(col), CHARINDEX(...)	LENGTH(col), STRPOS(...)
VARCHAR(MAX), NVARCHAR, DATETIME2	STRING, TIMESTAMP (or DATETIME — no timezone)

Two lab-specific notes:

- **The DDL is metadata-only.** Creating a table over external files registers a pointer; no data moves. It should return in a second or two. If your CREATE hangs or errors, the statement is wrong (typo in the URI, wrong location, missing connection) — the data is not "slow to load," because nothing loads.
- **No autodetect roulette here.** Yesterday's CSV autodetect pitfall (the N/A marker silently flipping a column to STRING) was a CSV problem. Parquet and Iceberg files carry their own schema, so types come out right by construction.

Full function reference when you need it: [GoogleSQL functions and operators](#).

Common gotchas

- **Sixteen people, one lab start time.** Seats and first queries can be slow right at 13:00 while everyone provisions and runs at once. Expect some waiting; don't refresh-spam or re-run in a loop.
- **Table-name collisions.** Everyone follows the same instructions against the same source. If you create a table with the exact name in the lab doc and a classmate's DDL shares your dataset, you'll collide. Suffix your table names with your initials or seat number (e.g., iceberg_orders_ia).
- **Copy-pasted URIs lose characters.** A gs:// URI that line-wraps in the lab doc can drop characters when pasted. The classic symptom: the table creates fine, then queries return 0 rows or "not found." If that happens, re-check the URI character by character against the lab doc.
- **Region mismatch.** The dataset's location and the bucket's location must be compatible. This produces the classic first-lab error message; the fix is to create your dataset in the location the lab specifies — don't freestyle it.
- **Query through the table, not the bucket.** If you find yourself browsing the GCS file listing to answer a question, come back to the SQL console — querying the files as a table is the entire exercise.
- **External table vs. Lakehouse (Iceberg) table, in one sentence:** this morning's external table was a read-only pointer you manage nothing about; a Lakehouse/Iceberg table is still files in GCS, but with table metadata and semantics on top — the path toward governance and ACID behavior. For your design: Bronze/Silver = Iceberg-family tables, not bare external tables.

If you finish early

- Run the same aggregate with and without a filter on the partition column; open **Job information / Execution details** and compare **bytes processed**. Then filter on a non-partition column and compare again. You are watching Iceberg's metadata pruning and predicate pushdown from the M9 slides.
- Deliberately write one query the T-SQL way (brackets, TOP, ISNULL), watch it fail, and fix it using the dialect box. Breaking it on purpose is the fastest way to learn the deltas.
- Browse the source bucket in the Cloud Storage console and find the Iceberg metadata (manifest) files next to the data files — that metadata is what the next 15-minute block is about.

Self-check

- ☐ Lab seat provisioned, and I confirmed I'm in the lab-provided project (project picker)
- ☐ Lakehouse table created over the existing Iceberg data, named with my suffix
- ☐ First SELECT returned rows — with no load job ever running
- ☐ I ran a filtered and an aggregated query and noted bytes processed
- ☐ I can say in one sentence how this differs from this morning's external table (Iceberg metadata adds table semantics over the same kind of files)

What this replaces in your current stack

The SSIS package + staging table you build today before anyone can ask a question of a file drop. Here, the file landing in the bucket *is* the queryable table. It also previews the end of linked-server/OPENQUERY patterns: you don't copy data next to the compute to query it.

Debrief: tie it to your Bronze design

Hold onto what just happened: data that was never loaded into BigQuery answered your SQL. Your Bronze zone is exactly this — raw landings in open formats on Cloud Storage, immediately queryable, immutable. The open questions — who can write these tables, who keeps the metadata, how this gets ACID guarantees and governance — are the next two blocks: [Iceberg explained](#) (13:45) and then the medallion architecture in [M10](#) (14:00), where the table you just made shows up as the Bronze zone in Google's own zone diagram.

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.